

Actividad 2.1 - Deteccion de carriles usando transformada de Hough

Portada

- Materia: Navegacion autonoma
- Actividad: Actividad 2.1 - Deteccion de carriles usando transformada de Hough
- Equipo: EquipoXX
- Integrantes: [Completar]
- Profesor: Dr. David Antonio Torres
- Fecha de entrega: 10 de mayo de 2026

Objetivo

Implementar en Webots un controlador en Python que detecte la linea amarilla central de la carretera mediante OpenCV y que convierta esa informacion en la entrada de un controlador PID para gobernar el angulo de direccion del vehiculo autonomo.

Entorno de trabajo

- Mundo utilizado: `city.wbt` de Webots R2025a o una copia derivada `city\_2023b.wbt`
- Robot controlado: `BmwX5` con nombre de seguimiento `vehicle` en la vista del mundo, segun configuracion de la clase
- Tipo de controlador: controlador del vehiculo en Python
- Velocidad objetivo de validacion: `50 km/h`
- Dependencias principales: `numpy`, `opencv-python`, API de Python de Webots

Descripcion de la solucion

La solucion propuesta sigue la secuencia indicada en la actividad. En cada paso de simulacion, el vehiculo obtiene la imagen de la camara a bordo y la convierte a escala de grises para simplificar el analisis. Posteriormente aplica el algoritmo de Canny para encontrar bordes intensos y construye una Region de Interes (ROI) trapezoidal usando `fillPoly`, con el fin de enfocarse en la mitad inferior de la imagen, donde se encuentra la informacion util del carril.

Sobre la imagen filtrada por la ROI se aplica la transformada probabilistica de Hough (`HoughLinesP`) para obtener una lista de segmentos rectos. A partir de esa lista se eliminan las lineas mayormente horizontales

para evitar falsos positivos provocados por cruces peatonales, intersecciones o ruido estructural del mundo. Para cada linea valida se calcula su punto medio horizontal y se compara contra el setpoint, definido como la mitad del ancho de la imagen.

El error entregado al controlador PID se obtiene seleccionando la linea cuyo punto medio horizontal tenga la menor distancia absoluta al setpoint. De esta forma, el sistema privilegia la referencia mas cercana al centro de vision del vehiculo. Si en un cuadro no se detecta ninguna linea valida, el error se fija en cero para que el vehiculo continúe recto hasta recuperar la marca amarilla.

Parametros principales

- Canny: `50` y `150`
- ROI trapezoidal:
 - `(0.18w, 1.00h)`
 - `(0.40w, 0.58h)`
 - `(0.60w, 0.58h)`
 - `(0.82w, 1.00h)`
- HoughLinesP:
 - $\rho = 1$
 - $\theta = \pi / 180$
 - $\text{threshold} = 20$
 - $\text{minLineLength} = 20$
 - $\text{maxLineGap} = 15$
- Filtro de lineas horizontales:
 - $\text{abs}(\text{slope}) < 0.3$
 - $\text{abs}(y_2 - y_1) < 5$
- PID inicial:
 - $K_p = 0.012$
 - $K_i = 0.0001$
 - $K_d = 0.006$

Controlador PID

El setpoint del controlador corresponde al centro horizontal de la imagen. Si la camara entrega una imagen de ancho 256 pixeles, entonces el setpoint es 128 pixeles. La salida del PID es el angulo de direccion del vehiculo, mientras que la velocidad se mantiene constante en `50 km/h`.

Las ganancias se eligieron como punto de partida para una sintonización manual en Webots:

- K_p se incrementa primero para que el vehículo reaccione ante el desplazamiento lateral.
- K_d se ajusta después para amortiguar la oscilación, especialmente en curvas.
- K_i se deja pequeño para corregir sesgos sostenidos sin introducir sobreoscilación.

Script entregado

El controlador principal se encuentra en el archivo:

- `sympy_controller_act_2_1.py`

Se adaptó tomando como referencia el sample vehicular de Webots que en la instalación local se encuentra en:

- `/opt/homebrew/Caskroom/webots/R2025a/Webots.app/Contents/projects/vehicles/worlds/city.wbt`
- `/opt/homebrew/Caskroom/webots/R2025a/Webots.app/Contents/projects/vehicles/controllers/autonomous_vehicle/autonomous_vehicle.c`

Pruebas realizadas

Escenario	Resultado esperado	Resultado observado
-----------	--------------------	---------------------

---	---	---
-----	-----	-----

Tramo recto con línea amarilla visible	El vehículo sigue la referencia sin oscilación severa	[Completar en Webots]
--	---	-----------------------

Curva con línea amarilla visible	El vehículo mantiene seguimiento razonable de la marca	[Completar en Webots]
----------------------------------	--	-----------------------

Entrada o salida de intersección	Cuando desaparece la línea, el vehículo no gira bruscamente	[Completar en Webots]
----------------------------------	---	-----------------------

Re-adquisición tras zona sin línea	El vehículo recupera el seguimiento al reaparecer la marca	[Completar en Webots]
------------------------------------	--	-----------------------

Resultados de sintonización PID

Parámetro	Valor final
-----------	-------------

---	---
-----	-----

K_p	[Completar]
-------	-------------

K_i	[Completar]
-------	-------------

| `Kd` | [Completar] |

Observaciones de la sintonización:

- [Completar]

- [Completar]

Video demostrativo

- Enlace de YouTube: [Pegar aqui]

Conclusiones

El controlador desarrollado integra un pipeline clásico de visión por computadora con un controlador PID para resolver el seguimiento de carril en un entorno simulado. La combinación de ROI, Canny y Hough permite concentrar el análisis en las zonas donde normalmente aparece la línea amarilla central, mientras que el PID transforma la geometría detectada en una acción de dirección interpretable por el vehículo.

La principal limitación del enfoque aparece en las intersecciones o en los tramos donde la línea amarilla no existe. Por ello se incorporó un comportamiento por omisión que manda conducir recto cuando no se detectan líneas válidas. Esto evita respuestas abruptas y facilita la recuperación del seguimiento una vez que la referencia visual vuelve a estar presente.

Declaración de uso de inteligencia artificial

OpenAI. (2026). ChatGPT/Codex, utilizado para apoyo en generación de código, estructuración del reporte y depuración conceptual. La validación final, ajuste de ganancias PID, generación del video y verificación del comportamiento en Webots deben ser realizados por el equipo autor de la entrega.

Anexo A. Script Python entregado

El siguiente código corresponde al controlador principal desarrollado para la actividad.

```
"""Actividad 2.1 - Deteccion de carriles con Hough + controlador PID en Webots.
```

```
Este controlador fue pensado para ejecutarse sobre el sample stock
`city.wbt` de Webots R2025a o sobre una copia derivada como `city_2023b.wbt`
usando el robot `vehicle` o el BmwX5 del mundo base. El flujo sigue
exactamente la secuencia solicitada en la actividad:
```

1. Captura de imagen desde la cámara a bordo.
2. Conversión a escala de grises.
3. Detección de bordes con Canny.
4. Aplicación de una región de interés (ROI) con fillPoly.
5. Detección de líneas con HoughLinesP.
6. Selección del error mínimo respecto al centro horizontal de la imagen.
7. Conversión del error a un ángulo de dirección usando un PID discreto.

```
Notas importantes para usarlo en Webots:
```

- Debe existir al menos una cámara en el robot.
- El mundo `city.wbt` stock trae un `Display` llamado `display`.
- Si agregas un segundo `Display`, este script puede mostrar imagen cruda y procesada por separado; si no, al menos muestra la vista procesada.
- Si los nombres de los dispositivos no coinciden con los candidatos definidos abajo, basta con ajustar las listas de nombres.
- La velocidad objetivo se mantiene en 50 km/h, como solicita la consigna.

```
from __future__ import annotations
```

```
import math
from dataclasses import dataclass
from typing import Iterable
```

```
import numpy as np
```

```
try:
```

```
    import cv2
except ImportError as exc: # pragma: no cover - depende del entorno Webots.
    raise SystemExit(
        "No se encontró OpenCV. Instala la dependencia 'opencv-python' en el "
        "Python configurado para Webots antes de ejecutar este controlador."
    ) from exc
```

```
try:
```

```
    from controller import Display
    from vehicle import Driver
except ImportError as exc: # pragma: no cover - depende del entorno Webots.
    raise SystemExit(
        "Este script debe ejecutarse desde el entorno de Webots con acceso a "
        "los módulos 'controller' y 'vehicle'."
    ) from exc
```

```
# Candidatos de nombres para facilitar el acoplamiento con el mundo recibido.
CAMERA_DEVICE_CANDIDATES = ("camera", "front_camera", "vehicle_camera")
RAW_DISPLAY_CANDIDATES = ("display_raw", "camera_display_raw", "raw_display")
PROCESSED_DISPLAY_CANDIDATES = (
    "display",
    "display_processed",
    "camera_display_processed",
    "processed_display",
    "display2",
)
```

```
# Parametros de la actividad.
TARGET_SPEED_KMH = 50.0
MAX_STEERING_ANGLE = 0.50
INTEGRAL_LIMIT = 3000.0
DEBUG_EVERY_N_STEPS = 10
```

```
# Parametros base de la detección de líneas.
CANNY_LOW_THRESHOLD = 50
CANNY_HIGH_THRESHOLD = 150
HOUGH_RHO = 1
HOUGH_THETA = np.pi / 180.0
HOUGH_THRESHOLD = 20
```

```

HOUGH_MIN_LINE_LENGTH = 20
HOUGH_MAX_LINE_GAP = 15
MIN_ABS_SLOPE = 0.30
MIN_DELTA_Y = 5

```

```

def clamp(value: float, lower: float, upper: float) -> float:
    """Limita un valor a un rango cerrado."""
    return max(lower, min(upper, value))

```

```

def try_get_device(driver: Driver, names: Iterable[str]):
    """Intenta obtener el primer dispositivo cuyo nombre exista en el robot."""
    for name in names:
        try:
            return driver.getDevice(name)
        except BaseException:
            continue
    return None

```

```

def camera_bgra_to_bgr(camera_image: bytes, width: int, height: int) -> np.ndarray:
    """Convierte la imagen BGRA de Webots a un arreglo BGR utilizable en OpenCV."""
    frame_bgra = np.frombuffer(camera_image, dtype=np.uint8).reshape((height, width, 4))
    return frame_bgra[:, :, :3].copy()

```

```

def bgr_to_display_bytes(image_bgr: np.ndarray) -> bytes:
    """Convierte una imagen BGR de OpenCV a bytes BGRA para pegarla en un Display."""
    image_bgra = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2BGRA)
    return image_bgra.tobytes()

```

```

def build_roi_polygon(width: int, height: int) -> np.ndarray:
    """Construye la ROI trapezoidal pedida en la actividad.

```

La ROI se limita a la mitad inferior porque ahí aparece la línea útil para la conducción inmediata. La parte superior contiene horizonte, edificios y elementos que agregan ruido a Canny y Hough.

```

    """
    return np.array(
        [
            [
                (int(0.18 * width), int(1.00 * height)),
                (int(0.40 * width), int(0.58 * height)),
                (int(0.60 * width), int(0.58 * height)),
                (int(0.82 * width), int(1.00 * height)),
            ]
        ],
        dtype=np.int32,
    )

```

```

@dataclass
class DetectionResult:
    """Contiene las salidas intermedias y finales del pipeline de vision."""

```

```

    processed_bgr: np.ndarray
    detected_segments: list[tuple[int, int, int, int]]
    usable_segments: list[tuple[int, int, int, int]]
    rejected_horizontal_segments: int
    setpoint: float
    chosen_error: float

```

```

def detect_lane_lines(frame_bgr: np.ndarray) -> DetectionResult:
    """Ejecuta el pipeline completo de detección de carriles.

```

La salida principal es la lista de líneas detectadas por Hough y el error elegido para alimentar el PID. El error se define como la distancia entre el punto medio horizontal de una línea y el centro horizontal de la imagen. Se usa la línea con el error absoluto más pequeño para seguir la marca amarilla más cercana al centro de la cámara.

```

    """
    height, width = frame_bgr.shape[:2]
    setpoint = width / 2.0

```

```

    gray = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, CANNY_LOW_THRESHOLD, CANNY_HIGH_THRESHOLD)

```

```

roi_polygon = build_roi_polygon(width, height)
mask = np.zeros_like(edges)
cv2.fillPoly(mask, roi_polygon, 255)
masked_edges = cv2.bitwise_and(edges, mask)

lines = cv2.HoughLinesP(
    masked_edges,
    rho=HOUGH_RHO,
    theta=HOUGH_THETA,
    threshold=HOUGH_THRESHOLD,
    minLineLength=HOUGH_MIN_LINE_LENGTH,
    maxLineGap=HOUGH_MAX_LINE_GAP,
)

processed_bgr = frame_bgr.copy()
cv2.polylines(processed_bgr, roi_polygon, isClosed=True, color=(255, 0, 255), thickness=2)

detected_segments: list[tuple[int, int, int, int]] = []
usable_segments: list[tuple[int, int, int, int]] = []
rejected_horizontal_segments = 0
candidate_errors: list[float] = []

if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = map(int, line[0])
        detected_segments.append((x1, y1, x2, y2))

        delta_x = x2 - x1
        delta_y = y2 - y1
        slope = math.inf if delta_x == 0 else delta_y / float(delta_x)

        # Se descartan lineas casi horizontales para no contaminar el error
        # del controlador con bordes de cruces peatonales o intersecciones.
        if abs(delta_y) < MIN_DELTA_Y or abs(slope) < MIN_ABS_SLOPE:
            rejected_horizontal_segments += 1
            cv2.line(processed_bgr, (x1, y1), (x2, y2), (0, 0, 255), 1)
            continue

        usable_segments.append((x1, y1, x2, y2))
        x_mid = (x1 + x2) / 2.0
        error = x_mid - setpoint
        candidate_errors.append(error)

        cv2.line(processed_bgr, (x1, y1), (x2, y2), (0, 255, 0), 2)
        cv2.circle(processed_bgr, (int(x_mid), int((y1 + y2) / 2.0)), 4, (255, 255, 0), -1)

if candidate_errors:
    # El error se obtiene de la linea util cuyo punto medio esta mas cerca
    # del centro horizontal de la imagen.
    chosen_error = min(candidate_errors, key=abs)
else:
    # Cuando no hay linea amarilla visible conviene mandar un angulo por
    # omision neutro: el vehiculo sigue recto hasta re-adquirir el carril.
    chosen_error = 0.0

cv2.line(
    processed_bgr,
    (int(setpoint), height),
    (int(setpoint), int(height * 0.55)),
    (255, 255, 255),
    2,
)

overlay_lines = [
    f"Hough total: {len(detected_segments)}",
    f"Lineas utiles: {len(usable_segments)}",
    f"Filtradas horizontales: {rejected_horizontal_segments}",
    f"Error seleccionado: {chosen_error:.2f}px",
]

for index, text in enumerate(overlay_lines):
    cv2.putText(
        processed_bgr,
        text,
        (10, 24 + index * 22),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.55,
        (255, 255, 255),
        1,
        cv2.LINE_AA,
    )

```

```

    )

    processed_preview = cv2.cvtColor(masked_edges, cv2.COLOR_GRAY2BGR)
    processed_bgr = cv2.addWeighted(processed_bgr, 0.70, processed_preview, 0.30, 0.0)

    return DetectionResult(
        processed_bgr=processed_bgr,
        detected_segments=detected_segments,
        usable_segments=usable_segments,
        rejected_horizontal_segments=rejected_horizontal_segments,
        setpoint=setpoint,
        chosen_error=chosen_error,
    )

@dataclass
class PIDController:
    """PID discreto sencillo para convertir el error visual en direccion."""

    kp: float
    ki: float
    kd: float
    integral: float = 0.0
    previous_error: float = 0.0

    def update(self, error: float, dt: float) -> float:
        """Calcula el angulo de direccion a partir del error actual."""
        proportional = self.kp * error

        self.integral += error * dt
        self.integral = clamp(self.integral, -INTEGRAL_LIMIT, INTEGRAL_LIMIT)
        integral_term = self.ki * self.integral

        derivative = 0.0 if dt <= 0.0 else (error - self.previous_error) / dt
        derivative_term = self.kd * derivative

        self.previous_error = error
        return proportional + integral_term + derivative_term

def paste_image_to_display(display, image_bytes: bytes, width: int, height: int) -> None:
    """Pega una imagen en un display de Webots si el dispositivo existe."""
    if display is None:
        return
    image_ref = display.imageNew(image_bytes, Display.BGRA, width, height)
    display.imagePaste(image_ref, 0, 0, False)
    display.imageDelete(image_ref)

def main() -> None:
    """Bucle principal del controlador externo."""
    driver = Driver()
    basic_timestep = int(driver.getBasicTimeStep())
    dt = basic_timestep / 1000.0

    camera = try_get_device(driver, CAMERA_DEVICE_CANDIDATES)
    if camera is None:
        raise RuntimeError(
            "No se encontro una camara. Revisa CAMERA_DEVICE_CANDIDATES y el "
            "nombre real del dispositivo en el mundo de Webots."
        )

    camera.enable(basic_timestep)
    width = camera.getWidth()
    height = camera.getHeight()

    raw_display = try_get_device(driver, RAW_DISPLAY_CANDIDATES)
    processed_display = try_get_device(driver, PROCESSED_DISPLAY_CANDIDATES)

    # Ganancias iniciales sugeridas en el plan. La idea es arrancar conservador
    # y afinar primero Kp, luego Kd y al final un Ki pequeno.
    pid = PIDController(kp=0.012, ki=0.0001, kd=0.006)

    driver.setCruisingSpeed(TARGET_SPEED_KMH)
    driver.setSteeringAngle(0.0)

    print("Controlador de la Actividad 2.1 iniciado.")
    print(f"Resolucion de camara: {width}x{height}")
    print(f"Displays encontrados: raw={raw_display is not None}, processed={processed_display is not None}")

```



```

print(
    "Sintonizacion inicial PID -> "
    f"Kp={pid.kp}, Ki={pid.ki}, Kd={pid.kd}, velocidad={TARGET_SPEED_KMH} km/h"
)

step_counter = 0
while driver.step() != -1:
    camera_image = camera.getImage()
    frame_bgr = camera_bgra_to_bgr(camera_image, width, height)

    detection = detect_lane_lines(frame_bgr)
    steering_angle = pid.update(detection.chosen_error, dt)
    steering_angle = clamp(steering_angle, -MAX_STEERING_ANGLE, MAX_STEERING_ANGLE)

    driver.setCruisingSpeed(TARGET_SPEED_KMH)
    driver.setSteeringAngle(steering_angle)

    # Si existen dos displays, el primero muestra la imagen cruda y el
    # segundo la imagen procesada. Si solo existe uno, se prioriza la
    # imagen procesada porque es la evidencia mas util para la actividad.
    if raw_display is not None and processed_display is not None:
        paste_image_to_display(raw_display, camera_image, width, height)
        paste_image_to_display(
            processed_display,
            bgr_to_display_bytes(detection.processed_bgr),
            width,
            height,
        )
    elif processed_display is not None:
        paste_image_to_display(
            processed_display,
            bgr_to_display_bytes(detection.processed_bgr),
            width,
            height,
        )
    elif raw_display is not None:
        paste_image_to_display(raw_display, camera_image, width, height)

    if step_counter % DEBUG_EVERY_N_STEPS == 0:
        print(
            "frame="
            f"{step_counter} "
            f"hough_total={len(detection.detected_segments)} "
            f"lineas utiles={len(detection.usable_segments)} "
            f"filtradas horizontales={detection.rejected_horizontal_segments} "
            f"error={detection.chosen_error:.2f} "
            f"steering={steering_angle:.4f}"
        )

    step_counter += 1

if __name__ == "__main__":
    main()

```